

LAB 2: AGILE BACKLOG CREATION & SPRINT SIMULATION IN JIRA

Aryaroop Bose - pes1ug24cs087

Section - B

Problem Statement - #02: Automated Rubric Assignment Evaluator

1. Objective & Scenario Overview

The objective of this laboratory session is to translate the functional and non-functional requirements identified in Lab 1 for Problem Statement #02 (Automated Rubric Assignment Evaluator) into a structured Agile Scrum backlog using Atlassian Jira Software. Requirements are structured into 4 major Epics, decomposed into 9 User Stories, estimated collaboratively via Planning Poker using the Fibonacci sequence, prioritized based on architectural dependencies, and simulated across two iterative Sprints to evaluate team velocity and burndown metrics.

2. Agile Hierarchy: Epics Specification

- **EPIC 1: Automated Submission & Test Execution Engine**
 - **Description:** Ingest uploaded project archives (.zip/.tar.gz), perform static syntax validation and dependency verification, execute pre-configured unit test suites in isolated sandbox containers with timeout enforcement, and compute baseline rubric scores.
 - **Mapped Lab 1 Requirements:** FR-001, FR-002, NFR-001, NFR-002
- **EPIC 2: Faculty Rubric Configuration & Evaluation Oversight**
 - **Description:** Provide faculty evaluators with an administrative interface to define weighted rubric criteria totaling 100%, review automated test execution results, and manually override automated scores with qualitative feedback and audit tracking.
 - **Mapped Lab 1 Requirements:** FR-003
- **EPIC 3: Anonymized Peer Review Management**
 - **Description:** Automatically redact student identifying data upon deadline closure and distribute submissions to designated peer reviewers according to allocation rules, enabling structured peer feedback.
 - **Mapped Lab 1 Requirements:** FR-004
- **EPIC 4: Student Feedback & Regrade Dispute Portal**
 - **Description:** Provide students with transparent visibility into itemized rubric score breakdowns, execution logs, and peer evaluations, and facilitate the submission of regrade requests during active dispute windows.
 - **Mapped Lab 1 Requirements:** FR-005

3. Product Backlog & User Stories Specification Table

- **ARE-5: Story 1.1: Archive Upload and Ingestion**
 - **Epic:** Epic 1
 - **Priority:** High
 - **Points:** 3
 - **Sprint:** Sprint 1
 - **Description:** As a student, I want to upload my project archive (.zip/.tar.gz), so that my submission is queued for automated evaluation.
- **ARE-6: Story 1.2: Static Syntax & Dependency Checks**
 - **Epic:** Epic 1
 - **Priority:** High
 - **Points:** 5
 - **Sprint:** Sprint 1
 - **Description:** As a system runner, I want to validate syntax and verify dependencies prior to test execution, so that broken or unauthorized packages are halted.
- **ARE-7: Story 1.3: Sandbox Test Suite Execution**
 - **Epic:** Epic 1
 - **Priority:** Highest
 - **Points:** 8
 - **Sprint:** Sprint 1
 - **Description:** As a system runner, I want to run unit test suites in an isolated sandbox container with a 15-second timeout, so that tests execute securely without resource exhaustion.
- **ARE-8: Story 2.1: Weighted Rubric Configuration**
 - **Epic:** Epic 2
 - **Priority:** High

- **Points:** 5
- **Sprint:** Sprint 1
- **Description:** As a faculty evaluator, I want to define weighted rubric criteria totaling 100%, so that automated grading aligns with course evaluation metrics.
- **ARE-9: Story 2.2: Manual Score Overrides & Remarks**
 - **Epic:** Epic 2
 - **Priority:** Medium
 - **Points:** 3
 - **Sprint:** Sprint 2
 - **Description:** As a faculty evaluator, I want to manually adjust automated scores and add qualitative remarks with an audit trail, so that subjective quality is evaluated.
- **ARE-10: Story 3.1: Anonymized Peer Allocation**
 - **Epic:** Epic 3
 - **Priority:** Medium
 - **Points:** 5
 - **Sprint:** Sprint 2
 - **Description:** As a faculty evaluator, I want submissions to be anonymized and distributed to peer reviewers upon deadline closure, so that peer grading remains unbiased.
- **ARE-11: Story 3.2: Peer Evaluation Submission**
 - **Epic:** Epic 3
 - **Priority:** Medium
 - **Points:** 3
 - **Sprint:** Sprint 2
 - **Description:** As a student peer reviewer, I want to submit rubric ratings and constructive comments on assigned submissions, so that peers receive structured feedback.
- **ARE-12: Story 4.1: View Rubric Breakdown & Logs**
 - **Epic:** Epic 4
 - **Priority:** High
 - **Points:** 3
 - **Sprint:** Sprint 2

- **Description:** As a student, I want to view my itemized rubric scores, execution logs, and peer comments, so that I understand my grade and execution errors.
- **ARE-13: Story 4.2: Regrade Dispute Workflow**
 - **Epic:** Epic 4
 - **Priority:** Low
 - **Points:** 2
 - **Sprint:** Sprint 2
 - **Description:** As a student, I want to submit a regrade dispute request with a mandatory rationale within the dispute window, so that grading errors can be rectified.

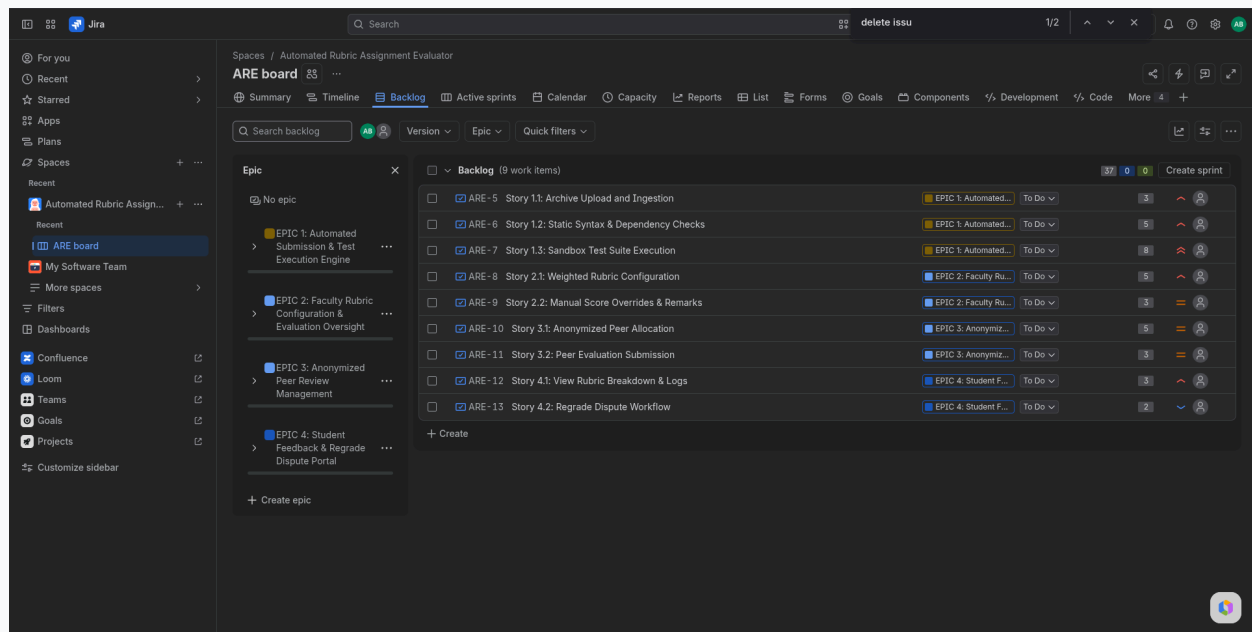


Figure 1: Jira Product Backlog with Epics, Stories & Story Points

4. Estimation Technique & Story Point Sizing

Story Points were estimated collaboratively via Planning Poker using the non-linear Fibonacci sequence (1, 2, 3, 5, 8, 13). The non-linear scale models increasing uncertainty and technical complexity across the backlog items:

- **ARE-7 (Sandbox Test Suite Execution - 8 Points):** Highest complexity due to security isolation, resource limits (1.5 GB memory limit), container teardown, and strict 15-second execution timeouts.
- **ARE-6 & ARE-8 (Syntax Checks & Weighted Rubric Config - 5 Points each):** Moderate-to-high complexity for AST syntax verification and multi-factor weighted rubric calculations.

- **ARE-5, ARE-9, ARE-11, ARE-12 (3 Points each):** Standard CRUD workflows for archive queueing, qualitative grading, and feedback visualization.
- **ARE-13 (Regrade Dispute Workflow - 2 Points):** Straightforward dispute submission form with active-window validation.

5. Sprint Simulation & Active Board Workflow Execution

The project was executed across two distinct, time-boxed Agile Sprints:

- **Sprint 1 (Target: Ingestion & Core Automated Evaluation):** Committed stories ARE-5, ARE-6, ARE-7, and ARE-8 totaling 21 Story Points. Stories transitioned through To Do to In Progress to Done, achieving 100% completion of committed core functionality.
- **Sprint 2 (Target: Peer Review, Student Portal & Dispute Resolution):** Committed stories ARE-9, ARE-10, ARE-11, ARE-12, and ARE-13 totaling 16 Story Points. Stories progressed through To Do to In Progress to Done, successfully concluding the full 37-point product backlog.

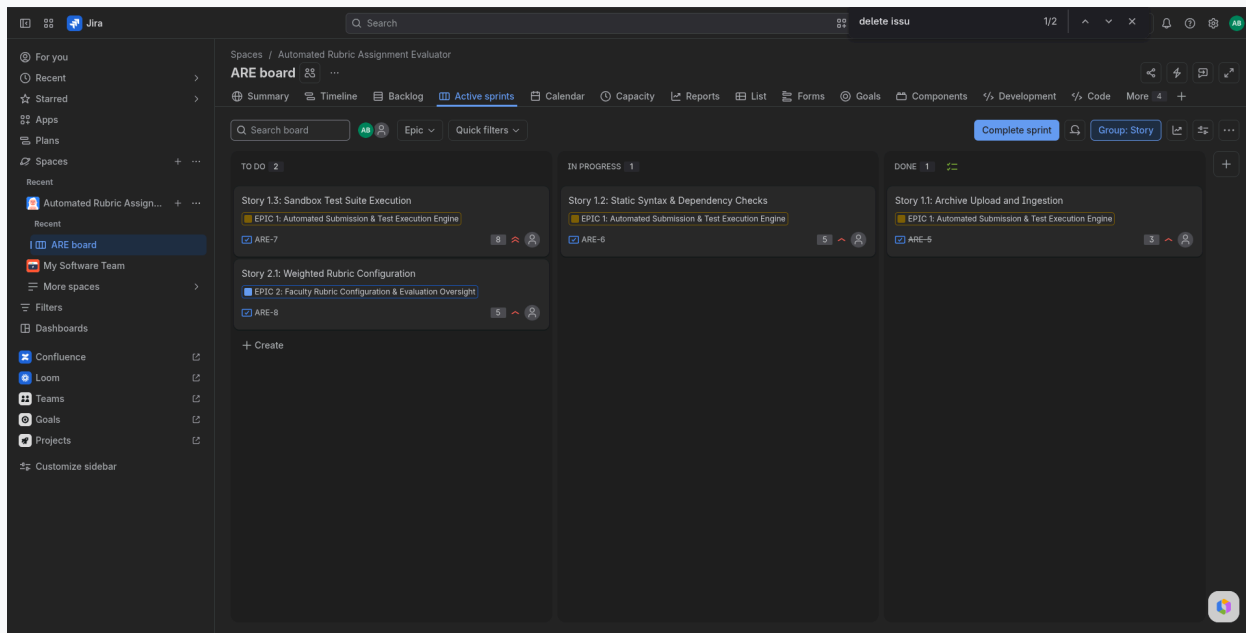


Figure 2: Active Sprint 1 Execution Board (To Do → In Progress → Done)

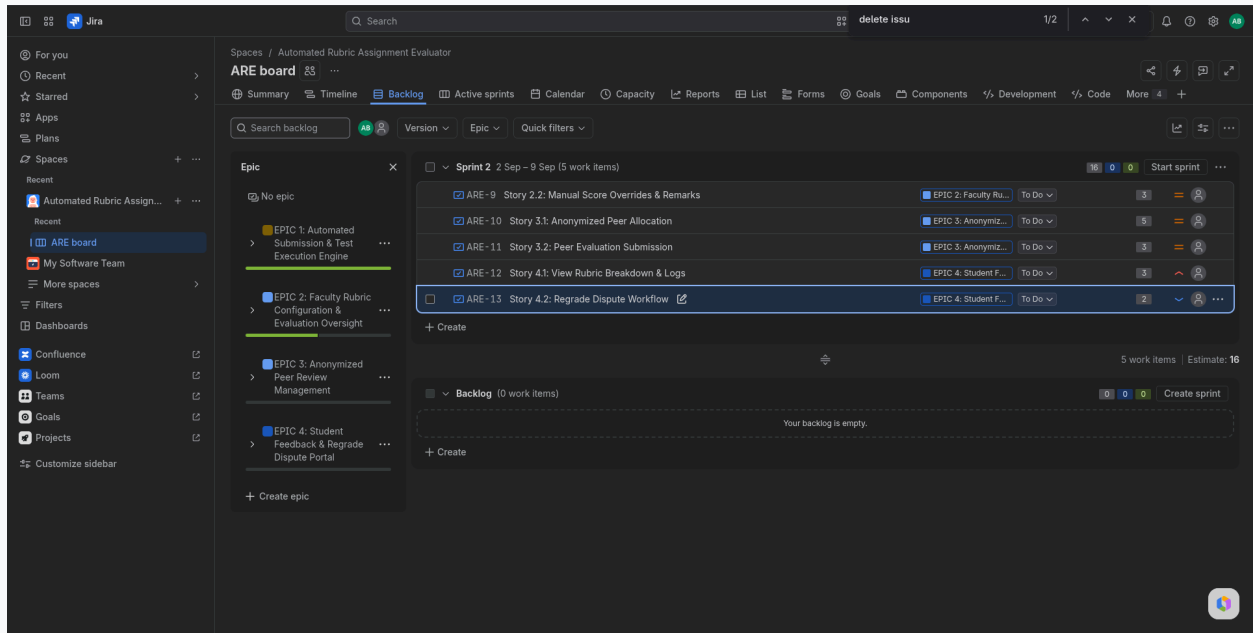


Figure 3: Sprint 2 Backlog Planning & Capacity Allocation

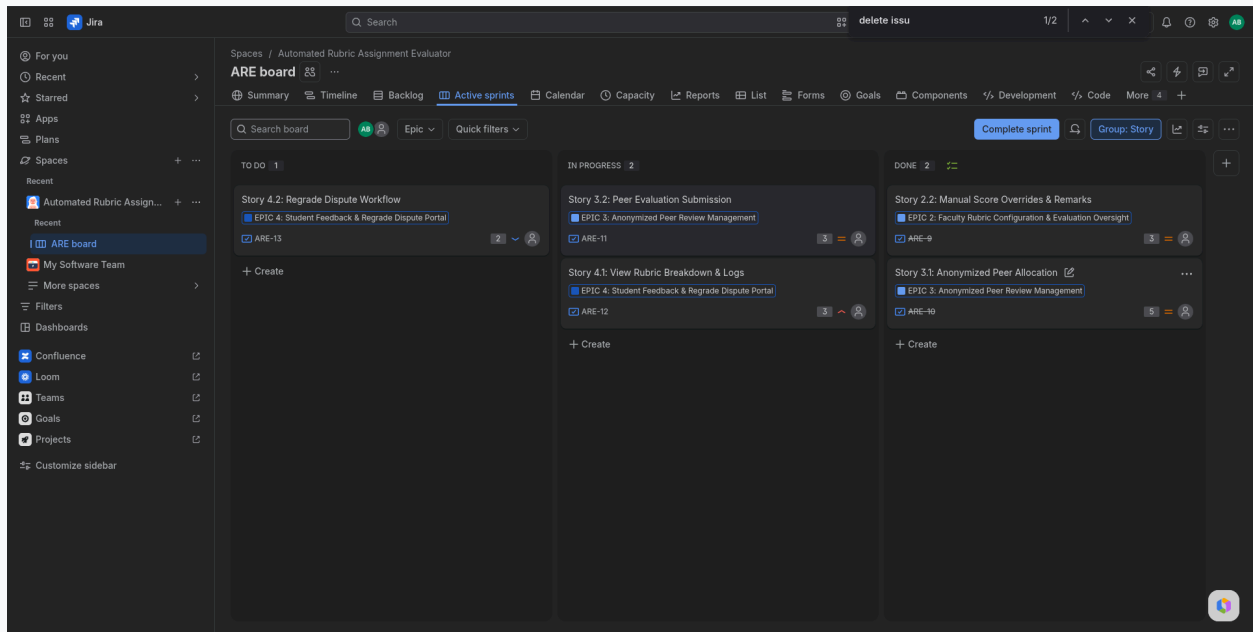


Figure 4: Active Sprint 2 Execution Board

6. Burndown Chart Analysis

Analysis of Jira's Burndown Reports for Sprint 1 and Sprint 2:

- Sprint 1 Burndown (21 Points):** Reflects initial scope addition (+21 points) at sprint activation followed by progressive burndown events (ARE-5: -3 pts, ARE-6: -5 pts, ARE-7: -8 pts, ARE-8: -5 pts), reaching 0 remaining points.

- **Sprint 2 Burndown (16 Points):** Shows initial commitment of 16 points along the guideline and sequential completion of ARE-9 (-3 pts), ARE-10 (-5 pts), ARE-11 (-3 pts), ARE-12 (-3 pts), and ARE-13 (-2 pts), achieving complete burndown.
- **Time-Compression Metric:** In a real-world multi-week sprint, tasks burn down gradually along the diagonal guideline. In this 60-minute laboratory simulation, work burned down within the compressed lab timeframe, confirming that 100% of committed work reached Done state.

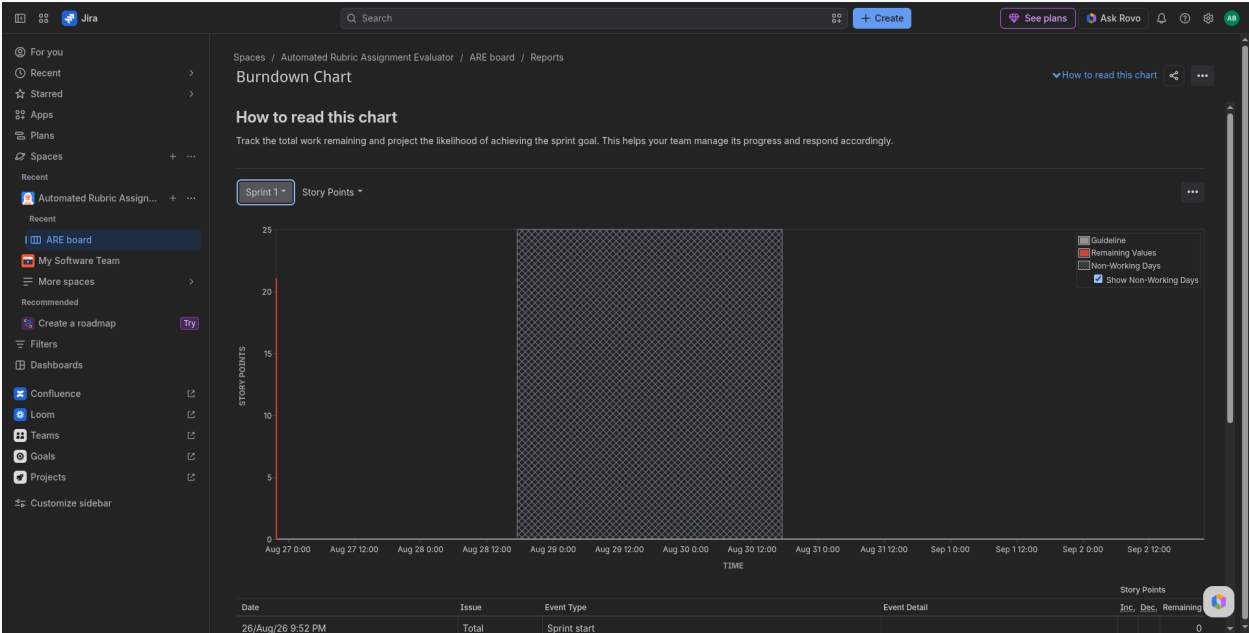


Figure 5: Sprint 1 Burndown Chart (Story Points Curve)

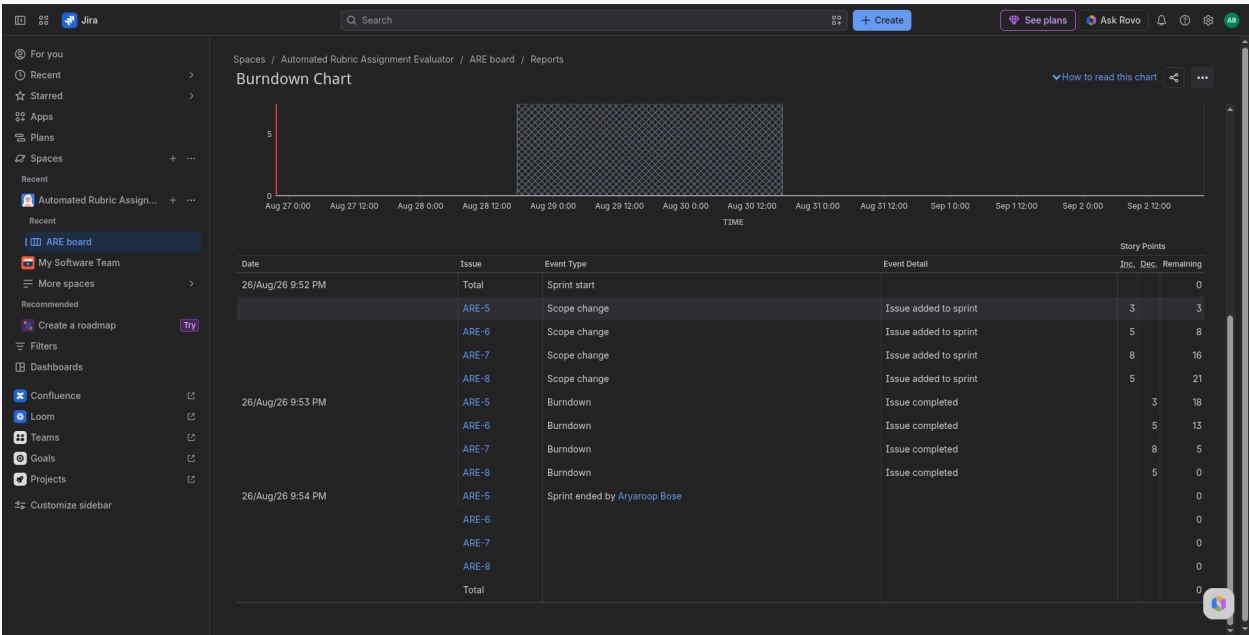


Figure 6: Sprint 1 Burndown Event History & Scope Log

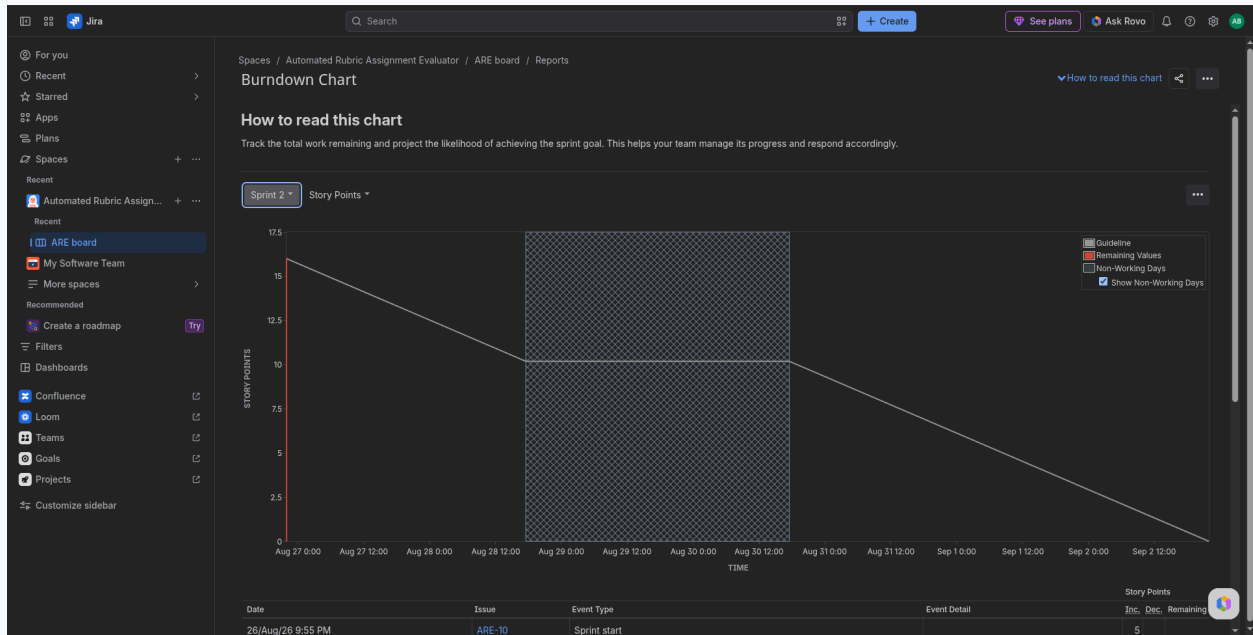


Figure 7: Sprint 2 Burndown Chart (Guideline & Remaining Work)

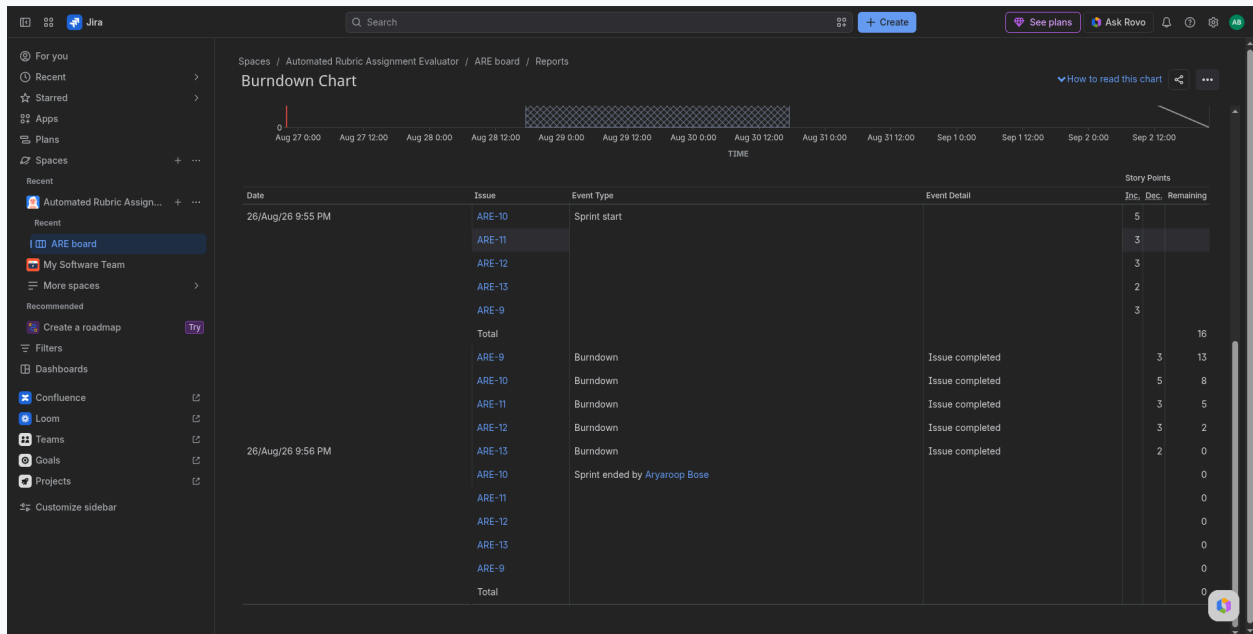


Figure 8: Sprint 2 Burndown Event History & Scope Log

7. Answers to Lab Reflection Questions

1. Did your estimations reflect the actual effort?

Yes. The relative Fibonacci estimations accurately captured system complexity. High-point stories such as the containerized sandbox runner (ARE-7, 8 pts) involved security isolation, timeout handling, and test-suite parsing, which require significantly more engineering effort than standard form inputs like the regrade dispute submission (ARE-13, 2 pts).

2. Was your backlog well-prioritized?

Yes. The backlog followed strict functional dependency ordering. Foundational infrastructure for file ingestion, syntax validation, and sandbox execution (Sprint 1) was prioritized at Highest/High priority, ensuring a testable automated evaluation core before building downstream dependencies like peer review distribution and student regrade appeals (Sprint 2).

3. How did your simulated sprint align with your plan?

The simulated sprint aligned directly with the release plan. Sprint 1 successfully achieved a velocity of 21 story points, completing all critical evaluation pipeline requirements. Sprint 2 delivered the remaining 16 story points, resulting in zero incomplete stories and zero spilled scope across the release.

4. What insights did the burndown chart give about your team's capacity?

The burndown chart confirmed that breaking large requirements into smaller, independently deliverable user stories (≤ 8 story points) provides clear visibility into progress. While the simulation executed rapidly, the chart demonstrated that the team's planned capacity of 21 points in Sprint 1 and 16 points in Sprint 2 was well-balanced and achievable without overburdening a single iteration.